

Prefix Sum

Definition: Used to perform multiple sum queries on subarrays efficiently. It involves precomputing an array where each element at index i stores the sum of all elements from the start up to i .

Sample Problem: Given an array `nums`, answer multiple queries about the sum of elements within a specific range $[i, j]$.

Example: `nums = [1, 2, 3, 4, 5, 6]`, range $[1, 3]$. Prefix Sum $P = [1, 3, 6, 10, 15, 21]$. Sum = $P[3] - P[0] = 10 - 1 = 9$.

Practice Questions:

[Range Sum Query - Immutable \(#303\)](#)

[Subarray Sum Equals K \(#560\)](#)

Two Pointers

Definition: Involves using two pointers that move through a data structure (usually a sorted array) toward each other or in the same direction to find a pair or subarray.

Sample Problem: Find two numbers in a sorted array that add up to a target value.

Example: `nums = [1, 2, 3, 4, 6]`, `target = 6`. Pointers at 1 and 6 (sum 7, too high), move right pointer to 4. $1 + 4 = 5$ (too low), move left pointer to 2. $2 + 4 = 6$. Output: $[2, 4]$.

Practice Questions:

[Two Sum II \(#167\)](#)

[Valid Palindrome \(#125\)](#)

[3Sum \(#15\)](#)

Sliding Window

Definition: Used to perform operations on a specific window size of an array or string. The window "slides" over the data to avoid redundant calculations.

Sample Problem: Find the maximum sum of a contiguous subarray of size k .

Example: $\text{nums} = [2, 1, 5, 1, 3, 2]$, $k = 3$. First window $[2, 1, 5]$ sum is 8. Slide to $[1, 5, 1]$ sum 7. Slide to $[5, 1, 3]$ sum 9. Max sum = 9.

Practice Questions:

[Maximum Average Subarray I \(#643\)](#)

[Longest Substring Without Repeating Characters \(#3\)](#)

[Minimum Window Substring \(#76\)](#)

Fast & Slow Pointers

Definition: Also known as Hare & Tortoise algorithm. Uses two pointers moving at different speeds to detect cycles or find the middle of a linked list.

Sample Problem: Detect if a linked list has a cycle.

Example: If a cycle exists, the fast pointer (2 steps) will eventually catch up to the slow pointer (1 step).

Practice Questions:

[Linked List Cycle \(#141\)](#)

[Happy Number \(#202\)](#)

[Find the Duplicate Number \(#287\)](#)

In-place Reversal of a Linked List

Definition: Reverses the nodes of a linked list without using extra memory by modifying the pointers.

Sample Problem: Reverse a linked list from position m to n .

Practice Questions:

[Reverse Linked List \(#206\)](#)

[Reverse Linked List II \(#92\)](#)

[Swap Nodes in Pairs \(#24\)](#)

Monotonic Stack

Definition: A stack that maintains elements in a specific order (increasing or decreasing) to find the next greater or smaller element.

Sample Problem: Find the next greater element for each element in an array.

Example: $nums = [2, 1, 2, 4, 3]$. Output: $[4, 2, 4, -1, -1]$.

Practice Questions:

[Next Greater Element I \(#496\)](#)

[Daily Temperatures \(#739\)](#)

[Largest Rectangle in Histogram \(#84\)](#)

Top 'K' Elements

Definition: Uses a Heap (Min-Heap or Max-Heap) to find the K largest or smallest elements in a collection.

Sample Problem: Find the Kth largest element in an array.

Practice Questions:

[Kth Largest Element in an Array \(#215\)](#)

[Top K Frequent Elements \(#347\)](#)

Overlapping Intervals

Definition: Involves sorting intervals and then merging or comparing them to find overlaps.

Sample Problem: Merge all overlapping intervals.

Example: `intervals = [[1,3],[2,6],[8,10]]`. Merged: `[[1,6],[8,10]]`.

Practice Questions:

[Merge Intervals \(#56\)](#)

[Insert Interval \(#57\)](#)

[Non-Overlapping Intervals \(#435\)](#)

Modified Binary Search

Definition: An extension of binary search used to solve problems in rotated sorted arrays or to find specific boundaries.

Sample Problem: Search for a target in a rotated sorted array.

Example: `nums = [4, 5, 6, 7, 0, 1, 2]`, `target = 0`. Output: 4.

Practice Questions:

[Search in Rotated Sorted Array \(#33\)](#)

[Find First and Last Position of Element \(#34\)](#)

Binary Tree Traversal

Definition: Techniques like BFS (Level-order) and DFS (Pre/In/Post-order) to visit all nodes in a tree.

Practice Questions:

[Binary Tree Level Order Traversal \(#102\)](#)

[Binary Tree Zigzag Level Order Traversal \(#103\)](#)

Depth First Search (DFS)

Definition: Explores as far as possible along each branch before backtracking. Common in tree and graph problems.

Practice Questions:

[Clone Graph \(#133\)](#)

[Path Sum II \(#113\)](#)

[Course Schedule II \(#210\)](#)

Breadth First Search (BFS)

Definition: Explores all neighbors at the present depth before moving to nodes at the next depth level. Ideal for shortest path in unweighted graphs.

Practice Questions:

[Rotting Oranges \(#994\)](#)

[Word Ladder \(#127\)](#)

Matrix Traversal

Definition: Treating a 2D matrix as a graph and using DFS or BFS to traverse it.

Practice Questions:

[Number of Islands \(#200\)](#)

[Flood Fill \(#733\)](#)

Backtracking

Definition: A recursive approach to find all (or some) solutions by building candidates and abandoning them ("backtracking") as soon as it determines they cannot lead to a valid solution.

Practice Questions:

[Permutations \(#46\)](#)

[Subsets \(#78\)](#)

[N-Queens \(#51\)](#)

Dynamic Programming (DP)

Definition: Solving complex problems by breaking them into simpler subproblems and storing the results (memoization or tabulation) to avoid redundant work.

Practice Questions:

[Climbing Stairs \(#70\)](#)

[House Robber \(#198\)](#)

[Coin Change \(#322\)](#)

